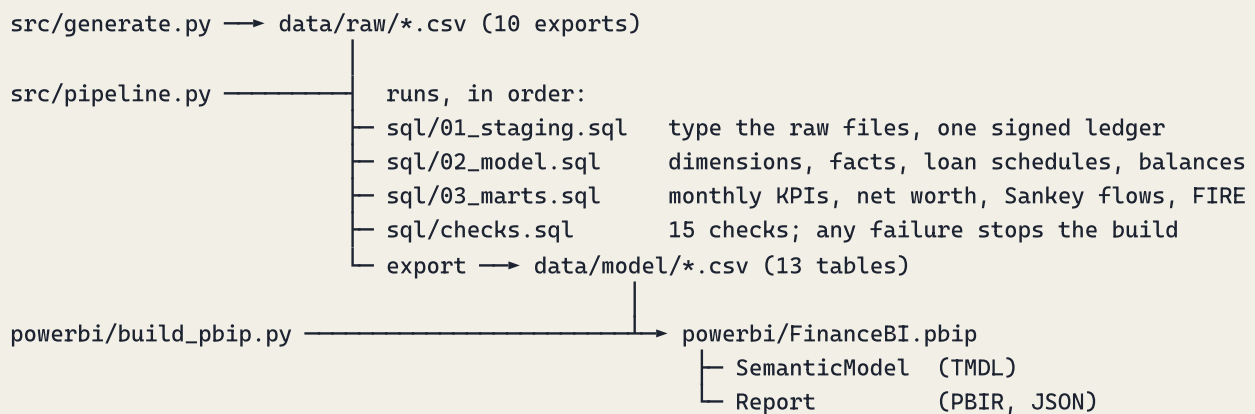


How it works

A walkthrough of the project for reviewers: what each part does, how data moves from raw exports to the report, how correctness is enforced, and where your eyes are most useful.

Companion to the case study. Paths are relative to the project folder.

1. The flow in one picture



Everything downstream of `generate.py` is deterministic. Same seed, same data, same numbers, same report.

2. Running it

```

python -m venv .venv
.venv\Scripts\pip install -r requirements.txt
.venv\Scripts\python src\generate.py          # writes data/raw
.venv\Scripts\python src\pipeline.py          # builds data/finance.duckdb, runs checks, writes
data/model
.venv\Scripts\python powerbi\build_pbip.py    # writes the Power BI project for your machine's
paths
  
```

Open `powerbi\FinanceBI.pbip` in Power BI Desktop and select **Refresh**. Run `build_pbip.py` first unless the project sits at `C:\personal-finance-bi`: it writes your local `data\model` path into the model's `DataFolder` parameter. The Sankey is Microsoft's free AppSource visual, loaded on first open.

Expected `pipeline.py` output: three "built" lines, 15 lines starting with `pass`, then "exported 13 tables".

3. Stage by stage

3.1 Generator: `src/generate.py`

Simulates one person month by month and writes what they could export from their bank, employer and investment platforms.

MECHANISM	HOW IT WORKS
Reproducibility	One seeded random generator (<code>SEED = 7</code>). Every amount, date and price comes from it.
Pay	CTC grows each April and jumps 35% at a July 2023 job switch. Gross, basic, EPF and TDS are derived per month; tax uses new-regime slabs by financial year (an approximation).
Spending	Fixed items on fixed days; discretionary totals are split across a random number of transactions with realistic merchant strings (UPI, POS, NACH, ATM). Inflation of 6% a year; Diwali and sale months spike.
Card	Card spend lands on the card statement and is paid from the bank on the 3rd of the next month, so card and bank must be reconciled, not added.
Loans	EMIs use the standard amortisation formula. The education loan predates the window, so only its tail appears.
Liquidity rule	SIPs always run. Optional moves (lump sums, PPF, FDs, REIT buys) only happen if the account stays above a ₹25k floor. Twice a year, cash above 1.5 months of committed spend is invested.
Self-check	The script refuses to write files if the bank balance ever goes negative.

3.2 Staging: `sql/01_staging.sql`

Reads each CSV with explicit types. The key output is `stg_transactions` : bank and card lines in one table with one sign convention (money in positive, money out negative). No business logic lives here.

3.3 Model: `sql/02_model.sql`

TABLE	BUILT HOW
<code>fact_transactions</code>	Every ledger line joined to <code>config/category_rules.csv</code> by regex; the lowest priority number wins. Unmatched lines would be "Uncategorized" (a check forbids any).
<code>fact_loan_schedule</code>	Recursive CTE: installment $k+1$ opens at installment k 's closing balance. The last installment clears the remainder exactly.
<code>fact_holdings</code>	Month-end value per instrument: cumulative fund and REIT units $\times$ that month's price; EPF and PPF from passbooks; FDs compound quarterly until closed; cash is the last bank balance in the month.
<code>fact_liabilities</code>	Loan outstanding from the schedule (full principal between disbursal and first EMI), plus the unpaid card balance.
<code>dim_month</code> , <code>dim_category</code>	The two dimensions every fact relates to. Months carry the Indian financial year label (April to March).

3.4 Marts: `sql/03_marts.sql`

MART	DEFINITION
mart_monthly	Income = take-home + bonus + REIT distributions + cashback. Spend = fixed + discretionary (accrual: card spend counts when it happens). Cash saved = income – spend – net investment. Savings rate = (income – spend) / income. Debt to income = EMIs / gross pay.
mart_net_worth	Assets by class (cash reserves, equity, fixed income, real estate) minus loans and card balance.
mart_cashflow_sankey	Source → target → amount rows in four stages: gross pay split; money in from all sources; split into fixed, discretionary, investments, cash; category detail. A month that drew down cash adds a "Cash drawn down" source so every node balances.
mart_fire_*	Inputs from the last 12 months (spend excluding EMIs and the car purchase, total contributions including EPF) and a 0–40 year baseline projection used to check the DAX.

3.5 Checks: `sql/checks.sql`

Each check is a query that returns the rows breaking a rule. `pipeline.py` runs them all, prints pass or fail, and exits with an error if any fail. The six most important:

CHECK	LOGIC
Balance chain	Each bank line's balance = previous balance + credit – debit.
Payroll	Salary credit in the bank = payslip net pay, every month.
Loans	Each month's EMI debit per loan = the schedule's EMI.
Card	Bill paid in month m = net card charges in month $m-1$.
Cash identity	Cash saved = change in bank balance – change in card balance. This ties the accrual view to the actual accounts.
Sankey balance	Into "Money in" = out of "Money in"; every parent node = sum of its children.

The checks were themselves tested: six deliberate corruptions inside a rolled-back transaction, each caught.

3.6 Power BI generation: `powerbi/build_pbip.py`

- **Model (TMDL).** Column types are read from the DuckDB schema, so the model can't drift from the data. Each table is a Power Query import from `DataFolder`. Calculated tables hold measures and what-if parameters (return, inflation, withdrawal rate, FIRE years).
- **Measures.** 30 DAX measures with format strings in lakh and crore. "Last month in context" measures (net worth, holdings) use the latest selected month so they work on cards and time axes alike. The FIRE projection uses the closed-form future value of a growing contribution stream, so no iteration is needed.
- **Report (PBIR).** Every page and visual is a JSON file. A shared rail (title, page navigator, synced period slicer) is stamped onto each page; theme colours, Sankey node colours and slider defaults are set explicitly.

3.7 Helper scripts: `powerbi/*.ps1`

SCRIPT	PURPOSE
<code>validate_tmdl.ps1</code>	Parses the model with Power BI Desktop's own TMDL serializer, offline. Catches format errors before Desktop does.
<code>query_desktop.ps1</code>	Runs a DAX query against the model open in Desktop (its local Analysis Services port). Used to match every KPI to DuckDB.
<code>reload.ps1</code>	Close Desktop, rebuild, reopen on a chosen page, trigger a data refresh, capture the window.
<code>capture_*.ps1</code>	Screenshot the Desktop window, or crop to the report canvas by detecting the rail colour.

4. Design choices worth challenging

CHOICE	WHY	TRADE-OFF
Synthetic data	Publishable, reproducible, still messy enough to need real cleaning.	Patterns are designed, not discovered.
Accrual spend	Card spend belongs to the month it happened.	Monthly "cash saved" differs from the bank balance change by the card float (the cash identity check makes this explicit).
EMIs in fixed spend	Conservative; matches how people budget.	Principal repayment is really saving, so the savings rate is understated in loan years.
CSV hand-off to Power BI	No ODBC driver or extra install; portable.	Types are re-declared in Power Query (generated from the DuckDB schema to stay in sync).
Generated PBIP	Reviewable diffs, reproducible report.	Formatting properties for some visuals are under-documented; a few were found in Power BI's own client code.
Deterministic FIRE	Simple to explain and verify.	No sequence-of-returns risk; a Monte Carlo band is the obvious next step.

5. What to review

1. **Correctness.** Do the mart definitions in section 3.4 match how you'd define savings rate, debt to income and net worth? Is anything double counted?
2. **Checks.** Is there a failure mode the 15 checks would miss?
3. **Model.** Is the star schema sensible for Power BI (relationships, hidden columns, measure table)?
4. **DAX.** Are the "last month in context" and FIRE measures clear and efficient?
5. **Report.** Does each page answer its question within a few seconds? Is anything misleading, redundant or hard to read?
6. **Story.** Are the case study's findings supported by the report?

6. Known limits

- Prices are simulated and tax is approximate; both are labelled.
- `DataFolder` is an absolute path (committed as `C:\personal-finance-bi\data\model\`); rebuild or edit it elsewhere.
- The report is built for Power BI Desktop; publishing to the Power BI service needs a work account.
- FY 2026-27 is a half year (April to September) and shouldn't be compared with full years.